RESEARCH AND IMPLEMENTATION NOTE FOR PROVEIT

One Implication Core, Two Proof Assistants

Further simplification of the K215 correctness proofs

21 October 2026

Abstract

The current repository already contains a 58-step representation-independent proof recovering unrestricted implication from the dominated row. The most useful next simplification is to reuse that proof more broadly. A separate Lean module still contains 106 modus-ponens steps deriving the ordinary implicational basis. We replace that second development by an adapter to the shared core and a five-step derivation of the missing S axiom. The two developments therefore use 63 shared modus-ponens steps instead of 164. Their maintained source shrinks from 42,715 to 18,668 bytes, including the new adapter. The pure Lean theorems have no axiom dependencies.

We also supply a checked Rocq implementation of the same generic core, its numerical compatibility adapter, and a checked Lean refactoring that separates semantic site contracts from the executable site checker. The latter is a necessary interface for replacing repeated per-occurrence execution by shared primitive proofs. It preserves the old Boolean-checker API and adds a simulation theorem which needs semantic contracts alone. All claims are pinned to an exact repository revision and separated from the full endpoint rebuild and primitive factoring work that remains to be done.

1 Scope, snapshot, and principal result

This note addresses proof length, conceptual duplication, and the structure of verification for the accepted K215 machine. It does not change its transition table, enumerator, boot sequence, or number of states. The source snapshot is ProveIt commit `2ca3a5c16f0876d2cab842056353c03a1ed2f5f0`. The review included the current Lean proof sources, the production Rocq extension, the phase-nine verification record, and the imported condensed-detachment work [1, 2, 4, 5].

The main recommendation is to maintain *one small implicational proof core* and two kinds of adapters: one for numerical formula codes with domination, and one for unrestricted implication with a trivial domination relation. The unrestricted adapter removes a large independent proof that still survives beside the recently simplified dominated proof.

Measured object	Before	After	Interpretation
Shared implication derivations	164 MP	63 MP	101 fewer MP applications
Lean logic sources	42,715 B	18,668 B	56.3% less source
Lean logic source lines	819	426	48.0% fewer lines
Rocq implication implementation	22,673 B	17,617 B	22.3% less source
Rocq implementation lines	1,169	365	68.8% fewer lines

The Lean comparison counts the original `DominatedImplication` and `Lukasiewicz` modules against the extended core, the new `ImplicationalBasis` adapter, and the retained `Lukasiewicz` bridge. The Rocq comparison counts the production `DomRho8.v` against the new generic core and numerical adapter;

the new core also exports K, S, and Peirce. These are source and derivation measurements, not estimates of total endpoint build time. They exclude audit clients and generation tools symmetrically; those remain in the archive for reproducibility.

The MP metric counts shared proof nodes in the mathematical derivations. The baseline has 58 MP nodes in the dominated derivation and 106 MP nodes after the axiom leaf of the older 107-declaration unrestricted derivation. The new proof retains all 58 nodes and adds five applications. It excludes the unchanged one-MP `lift_right` rule on both sides. Including that exported rule gives 165 to 64, with the same saving of 101 applications. This is a shared schema-derivation metric, not a minimum-proof claim.

1.1 The statement actually checked by the existing endpoint

The repository's precise external statement is

$$\mathrm{Halts}(K215, \mathrm{blank}) \quad \Longleftrightarrow \quad \text{there exists a finite ZFC derivation of falsity.}$$

Negating both sides gives nonhalting if and only if syntactic consistency. The Lean endpoint names are `blank_halts_iff_zfc_refutation` and `nonhalting_iff_zfc_consistency`; the Rocq dense endpoint gives the corresponding theorem pair [2, 10].

The verification record explicitly distinguishes these external theorems from an internal PA proof or an explicitly encoded ZFC derivation of the equivalence. This refactoring preserves that boundary. It supplies neither a new internalization argument nor a proof of consistency. In particular, removing axiom dependencies from the small implicational component does not remove the standard axioms from the full Lean endpoint.

2 Where the remaining proof cost lies

The endpoint itself is short because it composes independently verified layers: dense-state renaming, blank initialization, register-machine simulation, source/program correspondence, proof enumeration, and ZFC calibration [3, 10]. Shortening the final composition would accomplish little. The repeated developments and repeated local verification are better targets.

The pinned repository already incorporates several important reductions. Lean uses normalized source correspondence, a shared seed interpretation, a generic dominated-implication core, and a shared Home-return theorem. Phase nine retains all 4,751 source leaves and 6,176 branch outcomes. Its site batches took 984.608 seconds in the recorded run, versus 1,649.340 seconds in the historical phase-six run; the repository correctly labels this as a historical comparison, not a controlled paired benchmark [4]. None of these changes is claimed as a contribution of this note. The phase-nine endpoint checkpoint predates the latest 58-step core replacement; that replacement's commit reports focused compilation of the core, numerical adapter, and `KCodeDom`. Those two records must not be combined into a claim of a new full endpoint build.

Two useful opportunities remain:

1. **Logical sharing across modules.** The new dominated core and the old unrestricted `Lukasiewicz` development prove overlapping facts from the same implicational row. The overlap can be removed now.
2. **Semantic sharing across source occurrences.** The current simulation theorem consumes a proposition saying that each selected source edge passes `checkEdge`. A theorem describing an

instruction's complete behavior is not itself a Boolean-checker proof. A semantic interface is needed before shared instruction proofs can replace the repeated checks.

The first change is an immediate reduction. The second adds a small amount of infrastructure now and creates a precise route to a larger subsequent reduction. Treating these as separate changes avoids presenting a proposed future speedup as a measured result.

3 The representation-independent implication theorem

Let A be an arbitrary type, let $\Rightarrow : A \times A \to A$ be an arbitrary binary operation, and let $\preceq$ be a relation on A. No injectivity, freeness, formula decoder, arithmetic encoding, or truth constant is assumed. The `Language` interface requires exactly

$$a \preceq a, \tag{1}$$
$$u \preceq x \implies u \preceq (x \Rightarrow y), \tag{2}$$
$$u \preceq y \implies u \preceq (x \Rightarrow y). \tag{3}$$

Transitivity of $\preceq$ is not required. Define

$$\rho(a, b, c, u) = ((c \Rightarrow u) \Rightarrow a) \Rightarrow ((a \Rightarrow c) \Rightarrow (b \Rightarrow c)).$$

For $T : A \to \mathrm{Prop}$, the restricted proof interface consists of

$$T(x \Rightarrow y) \wedge T(x) \implies T(y), \tag{4}$$
$$u \preceq a \implies T(\rho(a, b, c, u)). \tag{5}$$

Theorem 3.1 (Existing shared core, with new basis exports)**.** *Under (1)–(5), every unrestricted instance $T(\rho(a, b, c, u))$ holds. The system also proves*

$$\mathsf{K}(a, b) = a \Rightarrow (b \Rightarrow a),$$
$$\mathsf{P}(a, b) = ((a \Rightarrow b) \Rightarrow a) \Rightarrow a,$$
$$\mathsf{B}(a, b, c) = (a \Rightarrow b) \Rightarrow ((b \Rightarrow c) \Rightarrow (a \Rightarrow c)),$$
$$\mathsf{S}(a, b, c) = (a \Rightarrow (b \Rightarrow c)) \Rightarrow ((a \Rightarrow b) \Rightarrow (a \Rightarrow c)).$$

The unrestricted row and B *are already outputs of the pinned 58-step proof;* K *and* P *are already intermediate results. Only* S *requires additional reasoning.*

The archive retains the exact existing condensed-detachment derivation. It does not replace it by a truth-table argument. Truth-table checks would establish classical validity, whereas the theorem needed here is derivability under an arbitrary predicate T satisfying the specified rules. The kernels check the actual applications of those rules.

The assumptions are useful in two substantially different instances. For the code instance, $A = \mathbb{N}$, $\Rightarrow$ = `wim`, and $\preceq$ is numerical $\leq$; the coding inequalities establish the two growth properties. For an unrestricted implication operation on any type, take $u \preceq a$ to mean True. The relation obligations then contain no mathematical content, and the unrestricted row supplies the restricted one directly.

4 A five-step extension that removes the second derivation

The new proof exposes K from the existing lemma `d12` and Peirce's law from `d18`. It also uses the existing formulas

$$\mathsf{E}(b,c) = b \Rightarrow ((b \Rightarrow c) \Rightarrow c), \qquad \texttt{d36},$$
$$\mathsf{R}(x,y,z) = (x \Rightarrow (y \Rightarrow z)) \Rightarrow ((y \Rightarrow x) \Rightarrow (y \Rightarrow z)), \qquad \texttt{d47}.$$

The numbering here refers to the generated 58-lemma module, whose source node identifiers need not be consecutive in the original derivation text.

Lemma 4.1 (Exchange, three MP applications)**.** *If $T(a \Rightarrow (b \Rightarrow c))$, then $T(b \Rightarrow (a \Rightarrow c))$.*

Proof. Put $D = (b \Rightarrow c) \Rightarrow c$ and suppose H proves $a \Rightarrow (b \Rightarrow c)$. The following are the three applications, in order:

1. Apply MP to $\mathsf{B}(a, b \Rightarrow c, c)$ and H. This proves $D \Rightarrow (a \Rightarrow c)$.
2. Apply MP to $\mathsf{B}(b, D, a \Rightarrow c)$ and $\mathsf{E}(b, c)$. This proves $(D \Rightarrow (a \Rightarrow c)) \Rightarrow (b \Rightarrow (a \Rightarrow c))$.
3. Detach the result of step 1 from the result of step 2.

This proves the asserted exchange rule without a deduction theorem, an assumption-discharge mechanism, or S itself. □

Theorem 4.2 (S from two MP applications and exchange)**.** *The existing core derives $T(\mathsf{S}(a,b,c))$ with five additional MP applications, counting the three in Lemma 4.1.*

Proof. Let

$$X = a \Rightarrow b, \qquad Y = (b \Rightarrow c) \Rightarrow (a \Rightarrow c), \qquad Z = (a \Rightarrow (b \Rightarrow c)) \Rightarrow (a \Rightarrow c).$$

The formula $\mathsf{B}(a,b,c)$ is $X \Rightarrow Y$. The formula $\mathsf{R}(b \Rightarrow c, a, c)$ is $Y \Rightarrow Z$. Detach these successively from $\mathsf{B}(X,Y,Z)$ to obtain $T(X \Rightarrow Z)$. These are two MP applications. Exchange its first two antecedents, namely X and $a \Rightarrow (b \Rightarrow c)$, to obtain

$$T((a \Rightarrow (b \Rightarrow c)) \Rightarrow ((a \Rightarrow b) \Rightarrow (a \Rightarrow c))).$$

There are no additional order side conditions. □

This derivation is useful because it reuses already-maintained lemmas. Searching for a shorter standalone proof of S could miss the smaller *combined* proof. The relevant optimization target is the union of the proof dependencies for unrestricted rho8, composition, prefixing, K, S, and Peirce.

4.1 What is and is not being claimed

The five-step extension and the removal of the independent 106-step proof are new refactorings relative to the pinned snapshot. The underlying implicational calculus, the existing 58-step proof, and the original K215 construction remain inherited work. There is no claim that the five-step derivation is globally novel in proof theory, or that 63 is an optimal node count. The improvement is a concrete reduction of this repository's proof dependencies, verified by the proof assistants.

5 Lean integration without changing the logical API

The new `ImplicationalBasis.lean` imports only `DominatedImplication`, which itself imports `Init`. It defines the original structure and exported theorem names in the original `Lukasiewicz` namespace. Its adapter uses

```
below := fun _ _ => True
refl := fun _ => True.intro
grow_left := fun _ _ => True.intro
grow_right := fun _ _ => True.intro
```

The row field merely reorders the parameters of the existing unrestricted `rho8` field. The four public theorems `k`, `peirce`, `composition`, and `s` become applications of shared-core theorems.

The original module becomes a bridge. It imports `TarskiComplete` and the new `ImplicationalBasis`. Its Tarski namespace tail is retained byte for byte. In particular, the proof of ex falso still uses the original `rho6` row and the derived K/S basis. The generic core does not silently acquire falsity or explosion as an extra axiom.

This split has a second benefit: the representation-independent basis no longer needs to import the full named-formula completeness development. Users requiring only an implicational basis can import the pure module directly. Existing users of `Lukasiewicz` retain the original bridge and declaration names.

The production numerical `DomRho8` adapter is unchanged by this Lean refactoring. The compatibility `truth` field remains in its public structure, although the generic theorem does not use it. This is deliberate API preservation rather than a mathematical requirement. An unrelated cleanup of that field would create more migration work than this proof reduction needs.

6 Rocq: the same mathematics behind a numerical adapter

At the pinned snapshot the production Rocq lane still uses the compact replay of an older certificate: 629 retained original nodes, 735 instructions, and 272 symbolic DAG entries in `DomRho8.v`. A 52-step numerical rendering is present as separate research; the repository's checked-boundary documentation does not describe it as the production proof [5, 8]. We compare against the production file, not against a discarded historical megabyte file. The existing 52-step numerical rendering is approximately 11,190 bytes, smaller than our 17,617-byte generic core plus adapter when only this one numerical theorem is considered. If isolated file size is the only objective, that specialized route deserves preference after its production checks. The benefit of our route is a common representation-independent library and a shared basis for multiple logical layers, rather than the smallest isolated rho8 proof.

The replacement consists of:

1. A generic `DominatedImplication.v`, implementing the same `Language`, restricted proof interface, 58-step derivation, and five-step basis extension as Lean. It has no imports and no numerical assumptions.
2. A small `DomRho8.v` compatibility adapter, preserving the original `DomImp` record fields and the public `rho8_all` theorem. The existing code inequalities discharge the growth properties.

The original `ThmDom.v` client is checked against the new adapter. The generic source is generated from the same pinned derivation text as Lean; the renderer produces explicit theorem applications and side-condition proofs. Thus the renderer and its unification algorithm are outside the trusted base. An

incorrect renderer produces a proof term that the kernel rejects.

6.1 Toolchain boundary

The production repository records Rocq 9.1.0 with rocq-stdlib 9.0.0. The available local checker is Coq 8.18.0. The generic core compiles directly there. For the numerical adapter and inherited client, the local reproduction copies use only the standard-library namespace spelling change `Stdlib` to `Coq`. The source snapshots, normalization record, commands, and logs are included. The adapter and client are also subjected to recursive `coqchk` in that local environment.

This supplies a substantive kernel check of the proposed mathematical replacement, not a fresh 9.1.0 production endpoint certificate. Adoption still requires the repository's pinned-toolchain build and regeneration manifest updates. The earlier recorded 237-source production proof is evidence for the baseline endpoint; it is not relabeled as a build of these replacement files.

7 A semantic interface for the expensive site proof

The current runtime proof obtains site correctness by evaluating `checkEdge` at each expanded source leaf. That Boolean check runs a symbolic executor with fixed fuel and verifies the exact target configuration. The executor is sound, but its acceptance proposition unnecessarily appears in the interface to the generic simulation theorem [9, 4].

The proposed `KSim` refactoring introduces `EdgeContract` and `SemanticCovered`. The checker remains a producer of semantic evidence:

$$\texttt{checkEdge}(M, Q, e) = \text{true} \quad \Longrightarrow \quad \texttt{EdgeContract}(M, Q, e).$$

The simulation proof itself consumes only the semantic evidence. A later shared instruction theorem can establish exactly the same proposition without proving that a particular evaluator accepts it within a particular fuel bound.

7.1 Complete contracts, including both decrement outcomes

Write $\mathrm{start}(Q)$ for the symbolic dispatch configuration and $\mathrm{conc}(n, v, s)$ for its concrete interpretation at gap n and register list v. The central proposition is

$$\begin{aligned}&\mathrm{RunsTo}(M, b, Q, s)\\ &\quad :\Longleftrightarrow \forall n\, \forall v,\ \mathrm{Valid}(b, v) \to \mathrm{Reach}^{+}_{M}(\mathrm{conc}(n, v, \mathrm{start}(Q)), \mathrm{conc}(n, v, s)).\end{aligned}$$

Here Reach^{+} requires at least one physical transition. The exact existing target configurations are retained. They account for successor paths, marker overflow, register position, and the extra blank cell introduced by decrement.

Source edge	Semantic obligation
Increment	Length/register guards and the complete updated-register target.
Decrement/test	The zero branch and the nonzero branch, each under its own validity condition, with its exact path and register effect.
Break	Length guard and the exact break-path target.
Continue	Length guard and the exact continue-path target.
Halt	Length guard and a reachable configuration at state 230.
Internal node	False; it cannot be accepted as an executable leaf.

The halt contract records arrival at the terminal port. The final simulation theorem separately assumes that both actions at state 230 are absent. This keeps the contract independent of a hidden terminal-table assumption.

The guards $|Q| \leq 48$ and $r < 9$ are inherited from the current simulation. The refactoring does not use an abstraction to discard inconvenient branch cases, path lengths, or register bounds. All gap and register quantifiers remain universal, with the same validity predicates as the existing checker soundness theorem.

7.2 Positive progress is logically necessary

Proposition 7.1 (Zero-step matching does not reflect halting)**.** *A relation matching every source transition by zero or more target transitions need not imply that target halting entails source halting.*

Proof. Take a one-state source whose only transition returns to itself and a one-state target with no transition. Relate the two states. Every source transition can be matched by zero target steps. The target is already halted; the source never halts. This example is also kernel checked in the archive's `ProgressBoundary.lean`. □

The existing reverse simulation argument performs strong induction on the remaining number of physical transitions. A matched source step must consume $j + 1$ target steps, so the induction measure decreases. The semantic refactoring preserves this requirement. A decoder prefix may legitimately have length zero; the *complete primitive outcome* must nevertheless make positive progress.

7.3 Coverage must remain indexed by actual occurrences

`SemanticCovered` quantifies over every source path P. It produces a successful source walk to an edge e, the consumed length d, and the complete contract at consume(P, d). This retains occurrence-specific coverage even if a single theorem supplies many instances.

The new `semanticCovered_of_allLeaves` theorem combines:

1. the equality between the program's source tree and the selected tree;
2. a purely structural traversal check using the constantly true leaf predicate; and
3. a semantic contract for every actual `LeafAt` occurrence.

Thus universal instruction lemmas can be combined with exact source membership. Classifying paths into a handful of instruction families without proving this membership and totality connection would leave a coverage gap.

7.4 Preserving the original interface

The new theorem `halts_iff_semantic` takes semantic coverage, the terminal-table condition, and the existing configuration relation. It does not take `BoxLemmas`, a symbolic executor, or an execution fuel. Those ingredients are used only when constructing contracts by the old checker.

The old theorem names and signatures remain available. The wrappers for `sim_step`, `sim_halt`, and `halts_iff` wrappers convert Boolean coverage to semantic coverage. The original direct `sim_stop` proof is retained, preserving its independence from `BoxLemmas`. Typed clients check these exact calling conventions. The executable definitions, including the checker, its fuel, starting configurations and targets, remain unchanged.

The local validation freshly compiles the 13-module import closure and an audit client. This checks the new semantic theorem and the compatibility API. It does not rerun the nine expensive K215 site batches or the full ZFC endpoint. No site has yet been removed, and no runtime speedup is attributed to this interface change alone. The interface increases `KSim` from 356 lines / 16,146 bytes to 480 lines / 21,995 bytes: 124 lines and 5,849 bytes of additional infrastructure. This cost is separate from the logical-source reductions in Section 1.

8 Verification and evidence discipline

The focused Lean logic build compiled 15 dependency modules, including the unchanged Tarski bridge, `Code254`, `KCode222`, `DomRho8`, and `KCodeDom`, followed by two audit clients. The runtime check compiled 13 dependency modules and one audit client. The Rocq candidate check compiled eight modules and recursively rechecked the typed client with `coqchk`. These dependency sets overlap; their counts should not be added to infer a full endpoint build.

The archive records the baseline revision, source hashes, generated outputs, compiler invocations, raw logs, and status for each local experiment. The important distinctions are:

- **Pure logical theorem checks.** The extended Lean core and the unrestricted basis have empty axiom reports. The Rocq generic exports are closed under the global context.
- **Compatibility checks.** The numerical Rocq client and the original Lean logical and runtime public APIs are checked against the replacements. No theorem is accepted by inserting a placeholder or a new axiom.
- **Reproducible generation.** The existing derivation text is retained; updated renderers regenerate the new files. Generated proof terms are checked by the kernel rather than trusted because their producer succeeded.
- **Production integration.** The packet is an integration proposal with checked components. A repository endpoint receipt must be regenerated after applying the chosen patches and rebuilding affected transitive descendants.

The compatibility claim is a claim about declaration types and proved mathematics. It is not a claim that old compiled objects can be combined with new source arbitrarily. The phase-nine history already records the danger of reusing byte-identical intermediate objects after a transitive executable change [4]. Source and import closure bindings should remain part of the adoption gate.

For the runtime interface, changing only proof terms leaves the checker definitions unchanged. Nevertheless a clean downstream check is still the appropriate way to publish an updated endpoint. For the logic refactoring, a new module and a changed import graph also require regeneration of the exact

dependency inventory. Source savings are not evidence that the original build closure remains the same size.

9 Recommended adoption sequence

9.1 First: share the logical basis

Apply the extended generic core, new pure basis module, and small Tarski bridge as one Lean change. Keep the public `Lukasiewicz` names and the existing numerical adapter. Regenerate the core with its synchronized producer, check the pure API audit, then rebuild the affected endpoint descendants and both closed endpoint clients. The numerical machine data and all site certificates should be byte-identical.

This change offers the clearest immediate benefit: an independent 106-step proof disappears, replaced by five applications using lemmas already maintained for the dominated proof. It does not depend on future decoder research.

9.2 Second: align the Rocq proof source

Add the generic core to the production Rocq build order before the numerical adapter. Update the generator, generated-source hashes, module list, and regeneration receipt together. Run the pinned 9.1.0 compile, inspect the `ThmDom` assumptions, and rerun recursive `coqchk`. The old certificate remains valuable historical evidence but no longer needs to be maintained as a second mathematical route.

The two proof assistants should share derivation data and theorem interfaces, not proof-assistant-specific compiled artifacts. Each kernel independently checks its own rendering. This arrangement keeps the generators untrusted and prevents the two languages from drifting into subtly different numerical normalization assumptions.

9.3 Third: introduce semantic site coverage

Adopt the `KSim` interface change while retaining the existing site checks and old theorem names. This isolates the API migration from the later replacement of any concrete obligation. Validate the unchanged endpoint through the compatibility wrappers.

Then choose one common primitive suffix and prove its full contract. Phase nine already identifies a source-only CB1 suffix design with 86 literal occurrences; it remains uncompiled research in that snapshot [4]. It is a plausible pilot, subject to its exact guard and frame conditions. Compose the checked decoder-entry theorem, the complete primitive theorem, and the existing Home return theorem. Discharge `EdgeContract` at the original full source paths.

Only after the pilot passes should its covered occurrences be removed from the expensive Boolean-execution batches. Measure a clean, paired build before and after that concrete replacement. Preserve the remaining obligations through the existing checker until their own contracts are proved.

10 Further research questions

1. **Optimize the combined proof graph.** Can a new condensed-detachment search targeting unrestricted rho8, K, S, Peirce, composition, and prefixing jointly improve on the shared 63-MP

construction? The old 58-step proof was optimized for a different collection of outputs. Count shared nodes and kernel cost as separate objectives.

2. **Expose a stable minimal basis interface.** Which of the internal evaluation, exchange, and composition lemmas should be public? A small named interface would make the five-step extension robust against renumbering of the generated proof without exposing every search-generated helper.
3. **Complete the first shared primitive contract.** Can the CB1 pilot, or another instruction family with a simpler frame invariant, discharge all of its actual occurrences using the new semantic API? The target should include both branch outcomes where applicable and positive physical progress.
4. **Use structural coverage certificates.** Can source-tree totality and leaf-family membership be represented in a compact shared DAG, while the semantic contract is proved once per family? The existing `LeafAt` interface supplies the exact proposition to preserve.
5. **Compare proof checking costs honestly.** How do direct theorem applications, a reflected compact certificate, and a shared semantic contract compare in source size, compiled object size, kernel conversion work, and clean build time? These measurements need not rank representations in the same order.
6. **Mirror the semantic interface in Rocq.** Can the same contract structure sit above Rocq's interpreter checks, so that primitive proofs can be shared conceptually between both proof lanes without changing either machine semantics?
7. **Internalize the final equivalence separately.** What explicit arithmetization and formal metatheory are needed to turn the existing external equivalence into the user's desired internal ZFC statement? The smaller pure implication proof makes one component easier to encode, but it does not settle the internalization of the full semantic calibration chain.

A Artifact guide and reproduction

The top-level `README.md` gives the exact packet layout and commands. The packet contains the article source and PDF, the Lean shared-basis replacement and generation tools, the Rocq generic core and numerical adapter, the semantic `KSim` patch and source closure, the progress counterexample, raw verification records, a source manifest, and patch files for review.

Compiled objects are omitted from the distributable archive. They are rebuilt from the supplied sources and the designated toolchain. The root verification script checks the manifest and static measurements; the separate proof drivers perform actual kernel checks. Passing the former is not represented as passing the latter. Full production reproduction continues to use the repository's existing endpoint drivers.

The source checkout is left unchanged. The packet's changes are staged as reviewable replacements and patches against the pinned revision. Because main is changing rapidly, applying them to a later revision should begin with a source comparison and preserve any intervening improvements.

B Trust and attribution

The inherited machine, calculus, compiler, code encodings, earlier derivations, and earlier verification are credited by the repository's K215 notice [11]. This packet retains that notice and the applicable source licenses. Its contribution is the additional cross-module proof sharing, the paired generic Rocq implementation, and the semantic site-contract interface, with the accompanying checks and integration material.

The proof-search and rendering programs are not trusted. The Lean compiler and kernel, the corresponding Rocq/Coq kernel and standard-library boundary, and their recorded versions are the local formal-check boundary. Source hashes and negative source scans help preserve artifact identity; they do not replace kernel checking or establish that an external metatheory has been internalized.

References

[1] ProveIt, pinned repository revision `2ca3a5c16f0876d2cab842056353c03a1ed2f5f0`. https://github.com/VladimirReshetnikov/ProveIt/commit/2ca3a5c16f0876d2cab842056353c03a1ed2f5f0

[2] ProveIt, *Formal correctness of the exact 215-state machine*, pinned snapshot, `Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/README.md`. https://github.com/VladimirReshetnikov/ProveIt/blob/2ca3a5c16f0876d2cab842056353c03a1ed2f5f0/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/README.md

[3] ProveIt, *Formal endpoints for the K-family candidates*, pinned snapshot, `Computability/BusyBeaver/Research/ZFC/k225-formal/README.md`.

[4] ProveIt, *Ninth K215 proof-simplification checkpoint*, pinned snapshot, `Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/phase9/README.md`.

[5] ProveIt, condensed-detachment derivation and producer, pinned snapshot, `Computability/BusyBeaver/Research/ZFC/k225-formal/rho8cd/`: `derivation_core.txt`, `generate_domrho8_cd.py`, `PROVEIT_STATUS.md`.

[6] ProveIt, generic dominated-implication proof, pinned snapshot, `Computability/BusyBeaver/Lean/BusyBeaver/ZFC/DominatedImplication.lean`.

[7] ProveIt, original unrestricted implicational development, pinned snapshot, `Computability/BusyBeaver/Lean/BusyBeaver/ZFC/Lukasiewicz.lean`.

[8] ProveIt, production compact certificate replay, pinned snapshot, `Computability/BusyBeaver/Research/ZFC/k225-formal/rocq/rocq-extension/coq/215/DomRho8.v`.

[9] ProveIt, generic site checker and simulation, pinned snapshot, `Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KSim.lean`.

[10] ProveIt, exact K215 endpoint, pinned snapshot, `Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KEndpoint215.lean`.

[11] ProveIt, K215 attribution notice, pinned snapshot, `Computability/BusyBeaver/Research/ZFC/candidate-k215/NOTICE.md`.